Data Structures (ETCCDS202)
Unit–1 Assignment: Foundations & Algorithmic Analysis

Name of the School: School of Engineering & Technology

Program: B. Tech CSE(AI & ML)

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Foundations & Algorithmic Analysis

Student Name: Abhay Kumar

Roll No.: 2501730185

Section: A

Submission Date: 05/03/2026

1. Abstract Data Type (ADT) and Stack

An Abstract Data Type (ADT) is a logical model of a data structure that defines what operations can be performed without specifying how the data is stored or implemented internally. ADTs focus on behavior rather than implementation.

In this assignment we implemented a Stack ADT. A stack follows the principle of Last In First Out (LIFO). This means the last element inserted into the stack is the first one to be removed.

Basic stack operations include:

Push: Adds an element to the top of the stack.

Pop: Removes the top element from the stack.

Peek: Returns the top element without removing it.

is_empty: Checks whether the stack is empty.

size: Returns the number of elements in the stack.

Stack operations usually take O(1) time because they operate only on the top element. There is no need to traverse the entire structure.

2. Recursion

Recursion is a technique where a function calls itself to solve smaller instances of the same problem. Every recursive function must have two key components: a base case and a recursive case. The base case stops the recursion, while the recursive case reduces the problem into a smaller subproblem.

3. Factorial Using Recursion

The factorial of a number n is defined as the product of all positive integers from 1 to n. Mathematically it is written as: $n! = n \times (n-1)!$.

The base case occurs when n equals 0 or 1 because both have factorial value 1.

Time Complexity: O(n)
Space Complexity: O(n) because each recursive call is stored in the call stack.
Big-O: O(n)
Big-Ω: O(n)
Big-Θ: O(n)

4. Fibonacci Sequence

The Fibonacci sequence is a famous mathematical series where each number is the sum of the two previous numbers.

In the naive recursive implementation, the algorithm repeatedly calculates the same values. For example, fib(5) recalculates fib(3) and fib(2) multiple times.

Naive Fibonacci Complexity:
Time Complexity: O(2^n)
Space Complexity: O(n)
This approach becomes very slow for large values of n.

Memoization improves the algorithm by storing previously computed results. When the function needs a value that has already been calculated, it simply retrieves it instead of computing it again.

Memoized Fibonacci Complexity:
Time Complexity: O(n)
Space Complexity: O(n)
This demonstrates the power of Dynamic Programming.

5. Tower of Hanoi

Tower of Hanoi is a classical problem used to demonstrate recursion. It consists of three rods and multiple disks of different sizes. The goal is to move all disks from the source rod to the destination rod while following specific rules.

The rules are:
1. Only one disk can be moved at a time.
2. A larger disk cannot be placed on top of a smaller disk.
3. Only the top disk of any rod can be moved.

The recursive idea is to move n−1 disks to the auxiliary rod, move the largest disk to the destination, and finally move the n−1 disks from the auxiliary rod to the destination.

The total number of moves required is 2^n − 1.
Time Complexity: O(2^n)
Space Complexity: O(n) due to recursion depth.

6. Recursive Binary Search

Binary search is an efficient searching algorithm that works on sorted arrays. It repeatedly divides the search space into two halves until the element is found or the search space becomes empty.

The recurrence relation for binary search is:
$T(n) = T(n/2) + O(1)$

Best Case: $O(1)$ when the middle element is the target.
Average Case: $O(\log n)$
Worst Case: $O(\log n)$

7. Algorithm Paradigms

Divide and Conquer

Divide and Conquer is a problem-solving strategy where a large problem is divided into smaller subproblems. Each subproblem is solved independently and their results are combined to obtain the final solution.

Binary search is a perfect example of Divide and Conquer. The algorithm divides the array into two halves and continues searching in the half that may contain the target element.

This approach significantly reduces the number of operations required because the problem size decreases rapidly.

Real-life Example: Searching for a word in a dictionary. Instead of checking every page sequentially, we open the book near the middle and decide whether to search the left or right half.

Dynamic Programming

Dynamic Programming is an optimization technique used when a problem contains overlapping subproblems. Instead of solving the same subproblem multiple times, the algorithm stores the solution and reuses it.

In the Fibonacci example, naive recursion recalculates the same Fibonacci numbers repeatedly. Memoization stores these results so that each Fibonacci value is computed only once.

This reduces the time complexity dramatically from exponential to linear.

Real-life Example: Suppose a student solves difficult math problems and writes down the solutions in a notebook. When similar problems appear in the future, the student can simply refer to the notebook instead of solving the problem again.

Greedy Algorithms

Greedy algorithms work by making the best possible choice at each step without considering future consequences. The idea is that a sequence of locally optimal choices can lead to a globally optimal solution.

Greedy algorithms are often used in optimization problems such as minimum spanning trees, shortest path algorithms, and coin change problems.

Real-life Example: When giving change in a store, the cashier usually selects the largest possible denomination first. This reduces the number of coins or notes used.

8. Conclusion

This assignment provided practical understanding of recursion, algorithm complexity, and algorithmic paradigms. Through the implementation of factorial, Fibonacci, Tower of Hanoi, and binary search, we explored how algorithms behave and how different paradigms can improve efficiency. Understanding these concepts is essential for designing efficient software systems.